Краткий разбор упрощённого проекта

Эта памятка относится к версии 3. Большой прежний PDF описывал сервер версии 2 с JobManager и опросом /api/status; для нового кода используйте этот документ.

Что нужно понять в первую очередь

Программа определяет тип текста, а не переписывает слова. Нейросети готовые, дообучения нет. Собственная часть - приложение, подготовка входа, вызовы моделей и сравнение с разметкой.

Модуль	Роль
app.py	Получить запрос и вернуть результат через Flask.
predict.py	Подготовить картинку и вызвать обе VLM.
evaluate.py	Сравнить ответы с разметкой, рассчитать и сохранить метрики.

Путь одного изображения

Файл в браузере -> FileReader/base64 -> POST /api/predict -> prepare_image -> ModelRunner.predict для 2B и Q3 -> JSON-ответ -> showAnswers.

Что стало проще

app.py: 193 -> 108 строк. Удалены самописные HTTP-классы, JobManager, callbacks progress/work и JavaScript poll. В root остались три основных Python-модуля; установка отделена в support/. Параметры моделей не менялись.

Почему страница не зависает

Браузер выполняет await fetch, блокирует повторные действия и показывает ожидание. Когда приходит ответ или ошибка, finally возвращает доступность кнопок. Сам сервер работает в одном потоке: модели не вычисляют две задачи одновременно.

Как готовиться

Прочитайте следующие три страницы, найдите model.generate, разберите 37/40 и два unknown. Затем покажите готовый печатный пример, рукописный и h12 с расхождением. Все 18 функций основного Python-кода описаны далее.

app.py: сервер и маршруты

Декоратор @app.get или @app.post связывает URL с обычной функцией. Flask выполняет обработку HTTP вместо собственных Handler-классов. Встроенный режим запуска используется локально, не как публичный промышленный сервер.

`read_data()`

Читает JSON-объект и разрешает POST только с того же Origin или без этого заголовка. Чужой Host отдельно отклоняется конфигурацией Flask. Вход - текущий HTTP-запрос; выход - словарь либо понятная ошибка.

`index()`

Отдаёт web/index.html. GET / -> страница приложения.

`config()`

Возвращает ключи моделей и список 50 примеров. GET /api/config -> JSON для начальной настройки интерфейса.

`report()`

Читает последний сохранённый JSON-протокол. GET /api/report -> готовые метрики и ответы; без файла - пустой отчёт.

`predict()`

Принимает base64-файл или sample_id, вызывает prepare_image, затем последовательно runner.predict для 2B и Q3. POST /api/predict -> сразу оба результата, без отдельной фоновой задачи.

`experiment()`

Вызывает run_evaluation с выбранным split и уже кэшированным runner. POST /api/evaluate -> полный отчёт эксперимента.

`image_file(name)`

Отдаёт JPG из data/images через безопасный send_from_directory. Другие расширения и выход из каталога получают 404.

`result_file(name)`

Отдаёт только JSON, CSV и HTML из results/. Используется ссылками на протоколы в интерфейсе.

`error_response(error)`

Преобразует ошибки в JSON с соответствующим HTTP-кодом. HTTPException сохраняет код, ValueError даёт 400, RuntimeError - 500.

`main()`

Разбирает порт и устройство, открывает браузер и запускает Flask на 127.0.0.1 в одном потоке, без debug и reloader. Один поток последовательно обслуживает запросы и исключает перекрытие вычислений.

predict.py: подготовка и нейросети

`parse_answer(text: str)`

Нормализует регистр, крайние пробелы и простую пунктуацию, затем проверяет точный ответ. Строка -> handwritten, printed, unknown или invalid. «This is printed» даёт invalid.

`prepare_image(data: bytes)`

Проверяет 15 МБ, формат и число пикселей, учитывает EXIF, белый фон прозрачности, RGB и большую сторону не более 1024. Байты -> PIL-изображение; невалидный файл -> ValueError.

`ModelRunner.__init__(self, device='auto')`

Запоминает device и создаёт пустой кэш loaded. Сами тяжёлые модели здесь ещё не загружаются.

`ModelRunner._load(self, key)`

Берёт модель из кэша или загружает локальные процессор и NF4-веса. Auto выбирает CUDA при доступности, иначе CPU. Возвращает процессор, модель, устройство, dtype и время загрузки. Повторный load_seconds равен 0.

`ModelRunner.predict(self, image: Image.Image, key: str)`

Собирает чат-шаблон, создаёт тензоры, вызывает model.generate, декодирует новые токены и проверяет ответ. Изображение и ключ -> label, raw, время, устройство и возможная техническая ошибка.

Главный фрагмент

```
with torch.inference_mode():
    generated = model.generate(
        **inputs, max_new_tokens=12,
        do_sample=False, use_cache=True
    )
```

Процессор подготавливает картинку и текст в формате модели. Генерируются новые токены; из декодирования исключается исходный вопрос. В модель не передаются имя файла, sample_id и правильный класс.

Что означают настройки

eval() - режим оценки; inference_mode - без градиентов; do_sample=False - без случайного сэмплирования. NF4 сжимает декодер, visual и lm_head подготовлены в FP16. На CPU обычные плавающие тензоры выбираются FP32. Это не обучение.

seconds включает загрузку при первом обращении и подготовку тензоров. generation_seconds охватывает генерацию и декодирование. load_seconds отдельно отражает первую загрузку; prepare_image вызывается до predict и в его время не входит.

evaluate.py: проверка и метрики

`calculate_metrics(truth: list, predictions: list)`

Строит матрицу 2 × 5 и считает метрики классов и сводку. truth и predictions -> Accuracy, Coverage, F1 macro, классы, матрица, числа отказов/сбоев.

`load_manifest(split='test')`

Читает CSV и выбирает строки dev либо test. Возвращает id, файл, метку, split и сведения происхождения; другой split отклоняется.

`run_evaluation(split='test', runner=None)`

Прогревает модели на dev, делает 80 измеряемых вызовов для test, чередует порядок моделей, сохраняет JSON и CSV. Возвращает report. Каждая неудача остаётся в данных и знаменателе Accuracy.

Формулы, которые нужно знать

Метрика	Объяснение
Accuracy	Верные ответы / все примеры. 37/40 = 92,5%.
Precision	TP/(TP+FP): доля верных ответов выбранного класса.
Recall	TP/(TP+FN): доля найденных истинных примеров класса.
F1	2PR/(P+R); F1 macro - среднее F1 двух классов.
Coverage	Доля ответов handwritten/printed, независимо от правильности.

Как объяснить Qwen2

Из 20 рукописных она нашла 17, один назвала печатным и на двух ответила unknown. Все 20 печатных названы правильно. Поэтому Accuracy=37/40, Coverage=38/40. Для рукописного класса Precision=100%, Recall=85%, F1≈91,9%.

Почему 100% - не гарантия

Набор небольшой и уже применялся в старой версии. Формулировки для моделей разные и выбраны по dev. Сравниваются настроенные связки на этих данных. Независимость авторов почерка и качество на пустых/смешанных кадрах не проверены.

Интерфейс и служебные файлы

Функции app.js	Назначение
$ / pct / seconds	Найти элемент, отформатировать процент и секунды.
error / setBusy	Показать ошибку; заблокировать или вернуть управление.
resetAnswers	Очистить предыдущие классы и сырые ответы.
showImage / selectFile	Превью, выбранный файл и ограничение 15 МБ.
api	GET или POST JSON через fetch, проверка HTTP-ошибки.
start	Ожидание ответа, вызов showAnswers/showReport, finally=setBusy(false).
showAnswers	Два класса, время, устройство, raw и согласие моделей.
cell	Создать текстовую ячейку таблицы.
showReport	Построить таблицы и матрицы по готовому JSON.
initialize	Загрузить config, dev-примеры и сохранённый отчёт.

Всего в app.js 14 именованных функций/функциональных переменных. События: выбор/перетаскивание файла, очистка, распознавание, эксперимент и выбор готового примера. FileReader.onload отправляет base64; onerror показывает сообщение и снимает busy. Отдельного опроса состояния больше нет.

Установка отдельно от основного алгоритма

Start_Windows.bat вызывает setup_windows.ps1. Его Run-Python проверяет код завершения. Установщик готовит отдельную .runtime, библиотеки и запускает support.download_models. Этот модуль проверяет SHA256 и скачивает официальные веса; support.quantize_models готовит NF4 без обучения.

tools/prepare_data.py - подготовка набора; tools/build_report.py - HTML и речь; tools/package_project.py - ZIP и CRC/SHA256. tests/ содержит 22 проверки. Эти инструменты не выполняются на каждом фото.

```
.runtime\python.exe -m unittest discover -s tests -v
.runtime\python.exe evaluate.py --split test
.runtime\python.exe app.py --device cpu
```

Показ и короткие ответы преподавателю

Порядок показа

1. Запустить заранее, прогреть модели на готовом примере.

2. Показать печатный и рукописный текст, раскрыть raw.

3. Открыть model.generate в predict.py и объяснить вход.

4. Показать 40 изображений, метрики и матрицы.

5. Загрузить h12: Qwen2 ошиблась, Qwen3 права.

6. Назвать ограничения и показать JSON/CSV.

Короткая речь

«В работе реализовано локальное приложение для определения типа текста. Две готовые VLM получают изображение и инструкцию. Обучения нет; приложение подготавливает вход, вызывает модели и сравнивает ответы с разметкой. После упрощения основной код состоит из трёх модулей. На повторной выборке из 40 фрагментов Qwen2 получила 92,5%, Qwen3 - 100%. Отказы учитываются как неуспешные ответы. Результат ограничен небольшой уже использованной выборкой и разными формулировками инструкций».

Где интеллектуальная часть?

В настоящем model.generate; класс не извлекается из имени файла.

Почему Precision 100%, а Accuracy ниже?

Все ответы «рукописный» верны, но часть рукописных пропущена; Recall=85%.

Что такое unknown / invalid / error?

Отказ модели / неверный формат генерации / техническое исключение.

Зачем NF4?

Сократить память без обучения; качество измеряется после сжатия.

Почему без платного Hugging Face?

Официальные веса выполняются локально. Это замена облачных Inference Providers, которую нужно обозначить при строгом требовании слайда.

Код: https://github.com/Sailats/3prac. Версия основного кода: f65ebdb. Протокол: test_20261024_160032.json.